

University of Puerto Rico
Mayagüez Campus
Mayagüez, Puerto Rico

Department of Electrical and Computer Engineering

Programming Assignment 1- Performance Comparison: Pure Python vs NumPy

by

Diego H. Merchan Rueda
Marco A. Monserrat Montoto
Edgard A. Díaz Bobé
Malik J. Paramo Rios

For: Professor Jose F. Vega Riveros
Course: ICOM5015, Section 001D
Date: February 25, 2026

Introduction	3
Purpose of Assignment	3
Hypothesis	3
Experimental Design Choice	3
Methodology	4
median_time	4
py_elementwise and np_elementwise	5
py_matmul and np_matmul	5
Results and Analysis	5
Conclusions	7
Lessons Learned	7
References	8
Group Collaboration	8

Introduction

Purpose of Assignment

Python is widely known as a programming language used in scientific computing due to how reliable it is and for its numerous applications and numerical libraries. However, since Python is a high-level computer language using explicit loops for numeric implementations can cause performance overhead. These limitations are covered by Numpy due to it being optimized for multidimensional arrays and vectorized operations. In this assignment pure python is implemented using explicit loops and Numpy is implemented using vectorized operations [2, 3]. Both had the task of making an element-wise multiplication of 1-D array and true matrix multiplication of 2-D array; With the goal to quantify runtime differences, evaluate scaling behavior, and explain the architectural reasons for performance gaps.

Hypothesis

We hypothesize that Numpy will outperform Python for 1-D element-wise multiplication, with a widening gap as the vector length increases. Additionally, we reasoned that Numpy will also outperform pure python in 2-D matrix multiplication, the speedup will increase as the matrixes grow in size. Matrix multiplication has a time complexity of $O(n^3)$ which would lead to interpreter overhead to affect pure python implementations at larger matrix sizes.

Experimental Design Choice

The experimental design comprises evaluating two fundamental numerical operations: the element-wise multiplication of one-dimensional arrays and the true matrix multiplication of two dimensional square arrays. In the pure python implementation, vector multiplication is carried out using a single iteration loop that multiplies said corresponding elements. The matrix multiplication is implemented using an explicit triple nested loop that follows the mathematical definition of computing each entry of the result as the dot product. In contrast, NumPy performs the same tasks, only that it uses a built-in optimized routines which handles the operations internally, not requiring manual iteration.

For the sake of fairness, both implementations operate on the same data generated for every input, ensuring that they are identical for each measurement. The experiment varies input size to observe how the performance scales, using vectors ranging from 10 to 500 elements and matrices from 10x10 up to 500x500. These sizes were selected to

illustrate how execution time increases as the problem becomes larger, allowing for a straight evaluation of how each approach handles the increasing computational demands.

Measurement Protocol:

- Warm-up run was performed per dimension as per requirements
- 15 trials were executed per test as required for reliability testing
- Median runtime was recorded for comparison
- Fixed random seed to ensure comparable results
- Correctness was verified using `np.allclose()`

Methodology

In this part it was decided to first use the libraries “*time*”, “*numpy*”, and “*matplotlib*”. Then we created five functions “*median_time*” to measure the time of execution by a function doing it various times, “*py_elementwise*” and “*np_elementwise*” are the functions in charge of multiplying the vectors (element to element), and “*py_matmul*” and “*np_matmul*” for matrix multiplication (2-D). This is for the principal goal of comparing the execution time of processes made by pure python or by the use of numpy library.

By having these functions the next step was to start the code for using the functions and compare the performance of runtime. Starting by adding a fixed random seed for reproducibility and then declaring the matrix/vector sizes which are “[10, 50, 100, 200, 500]” and creating two lists for each task (1-D and 2-D) to save the median of runtimes. Then the steps were applied to the 1-D and 2-D element-wise and matrix multiplication using the functions created inside a loop which goes across a list that contain the sizes (doing the task 5 times for each matrix/vector size). Inside the loop first matrices are created as a python list (using `random.rand`) and then those lists are converted as numpy arrays, then it goes the correctness verification using the “*numpy.allclose*” to verify the numerical equivalence within the tolerance [1]. Thereafter, comes the use of the “*median_time*” calling the functions “*py_elementwise*” and “*np_elementwise*” for the 1-D loop and “*py_matmul*” and “*np_matmul*” into the 2-D loop. Finishing with the data recollection of the measured times which then it showed in the notebook for logic reasoning. After all the process is saved and done, graphs are created to show the full data in a more user friendly way as asked in the assignment.

median_time

The goal of this function is to measure the time that takes a function to execute, doing it more than one time for performance comparisons. The inputs are “*func*” the function to benchmark (provided by user), “**args*” all arguments that will be passed into *func* (provided by user), “*trials*” number of timed trials to collect (15 by default, due to

measurement protocol which ask for a number of 10 to 20 trials), “*warmup*” number of untimed calls to stabilize the performance (1 by default) to accomplish one of the measurement protocols required. The output will be the *median runtime* across trials

py_elementwise and np_elementwise

The goal of these functions is to make the task of vector multiplication across pure python or the numpy library. The inputs for both are “*a*” and “*b*” which represent the two vectors for the element-wise multiplication. Either way, these functions are different because “*py_elementwise*” take the role of multiplying these vectors “ $a[i] * b[i]$ ” by using only python in a loop $O(n)$, and saving it in a list as a result, representing a nested loop. Meanwhile, “*np_elementwise*” make the same multiplication but with the python library NumPy using “ $a * b$ ”. All of this with the purpose of later comparing the runtime performance of these two actions.

py_matmul and np_matmul

These functions do the same task as the ones before (*py_elementwise* and *np_elementwise*) but with the task of matrix multiplication besides vector multiplication. Here the inputs are the same “*A*” and “*B*” representing the matrix to multiply. The outputs are the matrix given by the multiplication, let's say “*C*” matrix. The change here is how the multiplication is done by each function, “*py_matmul*” used a three nested loop $O(n^3)$ this is because for a matrix multiplication the first loop must go through each row of “*A*”, each column of “*B*” at the second loop, to give us the at the third loop the product “*C*”. Then “*np_matmul*” calculates this by using the operator “*@*” giving the product directly.

Results and Analysis

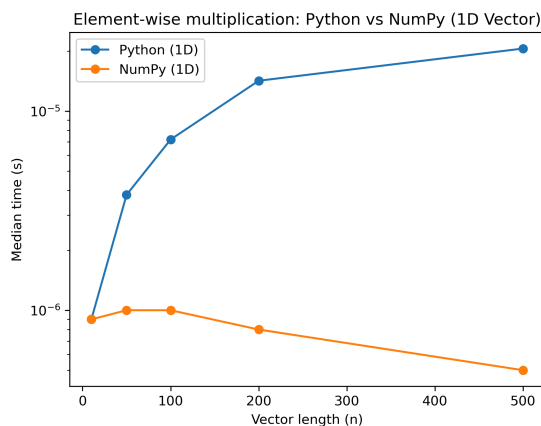


Fig.1. Graph 1-D Vector Runtime Performance

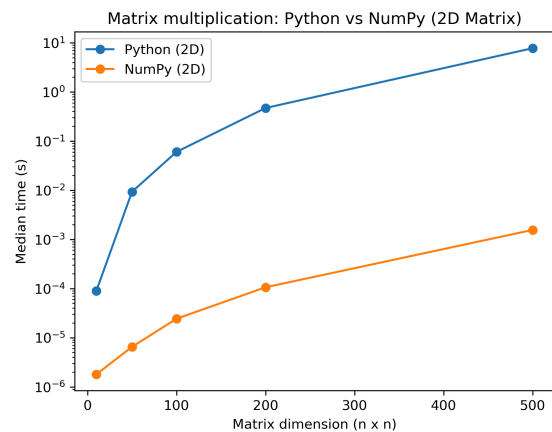


Fig.2. Graph 2-D Matrix Runtime Performance

Figure 1 and Table 1 summarize the median runtime for element-wise multiplication using a pure Python loop and NumPy's vectorized multiplication. As vector length increases, runtime increases as expected, since this operation scales roughly linearly with the number of elements. For very small vectors, both methods run in the microsecond range, so fixed overhead such as timer resolution and function-call costs can partially mask differences. However, as the vectors grow, NumPy's advantage becomes increasingly clear.

n (longitude)	Python(s) median	NumPy(s) median	Speedup (Tpy/Tnp)
10	$1.00 * 10^{-6}$	$9.00 * 10^{-7}$	1.11
50	$3.60 * 10^{-6}$	$9.00 * 10^{-7}$	4.00
100	$6.60 * 10^{-6}$	$8.00 * 10^{-7}$	8.25
200	$1.29 * 10^{-5}$	$1.00 * 10^{-6}$	12.90
500	$3.34 * 10^{-5}$	$8.00 * 10^{-6}$	33.40

Fig.3. Table 1 (Element-wise multiplication 1D)

Figure 2 and Table 2 present the median runtime for true matrix multiplication using pure Python triple nested loops versus NumPy's matrix multiplication. Figure 2 plots runtime versus matrix dimension for both approaches on the same axes, enabling direct comparison of scaling behavior. Because runtimes span a wide range, the y-axis is shown on a logarithmic scale. The performance gap becomes extremely large as matrix dimension increases. Pure Python runtime grows rapidly with size, while NumPy remains in the millisecond range even for larger matrices. This is consistent with the fact that matrix multiplication requires a number of arithmetic operations that grows in proportion to the cube of the matrix dimension, so interpreter overhead compounds quickly as matrices get larger.

Matrix size	Python (s)	NumPy (s)	Speedup
10x10	$1.229 * 10^{-4}$	$2.10 * 10^{-6}$	58.52
50x50	$1.5159 * 10^{-2}$	$9.80 * 10^{-6}$	1546.84
100x100	$1.216171 * 10^{-1}$	$5.53 * 10^{-5}$	2199.22
200x200	$9.474391 * 10^{-1}$	$4.463 * 10^{-4}$	2122.88

500x500	$1.82173 * 10^1$	$2.4997 * 10^{-3}$	7287.80
---------	------------------	--------------------	---------

Fig.4. Table 2 (Matrix multiplication 2D)

NumPy is faster primarily because its core operations are executed in compiled, highly optimized code. In contrast, the pure Python implementations perform arithmetic inside Python-level loops, where every iteration adds interpreter overhead from loop control, repeated indexing, and dynamic type handling. That overhead becomes especially costly in matrix multiplication because the total number of loop iterations grows very quickly as the matrix size increases. Memory layout also contributes to the difference: NumPy arrays store numeric data contiguously in memory, improving cache efficiency and enabling low-level optimizations, while Python's list-based structures rely on pointer indirection and poorer spatial locality, increasing memory-access overhead in large nested-loop computations.

Conclusions

Throughout this assignment pure Python implementations were tested against NumPy implementations for vector and matrix multiplication. Results showed a very clear margin in performance between the two implementations with NumPy outperforming pure Python as input size increased. While both completed their task and achieved correct values, they did not achieve it at the same speed. Pure python used explicit loops which became increasingly slower as the data grew in size due to computational overhead. In comparison, Numpy used low-level libraries and vectorized equations leading to a faster and more efficient code execution. This experiment led to understanding that matrix multiplication has a much larger speed difference than vector multiplication and that vectorization is necessary for scalable numerical computing. Overall, NumPy is the better tool of choice when compared to standard Python loops for numerical operations.

Lessons Learned

Complexity matters when deciding implementations, using pure python with time complexities like $O(N^3)$ is inefficient. Using the right optimized libraries helps reduce practical runtimes. Benchmarking methodology is necessary, methods like warm-up runs, multiple trials, median timing and fixed random seed help ensure an accurate and fair comparison. Scalability demonstrates performance better, since inefficient codes get slower the bigger the scale. Correctness must be verified, before testing performance speed verifying that the results are equivalent is important for scientific computing.

References

- [1] NumPy Developers, “numpy.allclose,” *NumPy v2.1 Manual*, 2024. [Online]. Available: <https://numpy.org/doc/2.1/reference/generated/numpy.allclose.html>. [Accessed: Feb. 24, 2026].
- [2] Python Software Foundation, *Python 3 Documentation*. [Online]. Available: <https://docs.python.org/3/> [Accessed: Feb. 24, 2026].
- [3] NumPy Developers, *NumPy Documentation*. [Online]. Available: <https://numpy.org/doc/> [Accessed: Feb. 24, 2026].

Group Collaboration

1. Diego - Created the function for median_time which connected the other functions created to record their execution time.
2. Malik - Created the functions Py_elementwise, np_elementwise for the multiplication of vectors(1-D).
3. Edgard- Created the functions py_matmul and np_matmul for the multiplication of matrix (2-D).
4. Marco - Used created functions to obtain results and do their plot. Worked in Introduction and Conclusion.

Once we obtained the results, our group of four analyzed the graphics together to compare them with our initial hypothesis and came up with the final conclusion. In this same manner, we obtained the lessons learned and started the report.

Una vez ya se obtuvieron los resultados, los cuatro de nosotros decidimos analizar juntos la gráficas para analizar la hipótesis que tuvimos en un principio y concluir que obtuvimos al final. De esta manera obtuvimos las lecciones aprendidas y pasamos a hacer el reporte.